Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №7
на тему
«ЗАЩИТА ПО ОТ НЕСАНКЦИОНИРОВАННОГО ИСПОЛЬЗОВАНИЯ»

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

_____
(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Классификация методов обфускации 4
2.2 Использованные стратегии обфускации 4
2.3 Реализация обфускатора 5
2.4 Тестирование и результаты 6
Заключение 7
Приложение А (обязательное) Листинг программного кода 8

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение методов защиты программного обеспечения от анализа и несанкционированного использования с помощью обфускации. Задачи включают изучение теоретических основ обфускации, разработку программного средства для автоматического преобразования исходного кода и анализ эффективности примененных методов защиты.

Конечным результатом работы является обфускатор на языке Python, реализующий методы переименования идентификаторов и кодирования строковых литералов.

2 ХОД РАБОТЫ

В ходе выполнения работы было разработано программное средство, выполняющее лексические и синтаксические преобразования исходного кода Python. Для работы с кодом использовалась библиотека ast (Abstract Syntax Trees), которая позволяет безопасно модифицировать структуру программы без риска нарушения синтаксиса.

2.1 Классификация методов обфускации

Методы обфускации могут быть классифицированы по уровню и объекту воздействия на программный код. В современной практике выделяют следующие категории:

1 Лексическая обфускация (Layout Obfuscation). Включает в себя преобразования, которые изменяют только внешний вид исходного текста, не затрагивая его логическую структуру. Основные приемы: удаление комментариев, минимизация (удаление пробелов и переносов строк) и переименование идентификаторов. Данный метод реализован в рамках текущей лабораторной работы.

2 Обфускация данных (Data Obfuscation). Направлена на скрытие реальных значений и структур данных. Примеры включают кодирование строковых литералов (использовано в работе), разбиение переменных на части, изменение способов индексации массивов и преобразование простых констант в сложные вычислимые выражения.

3 Обфускация потока управления (Control Flow Obfuscation). Изменение логической структуры программы для затруднения её анализа. Ключевые методы: использование непрозрачных предикатов – условных переходов, результат которых всегда постоянен, но трудно определим статически, выравнивание потока, при котором иерархическая структура программы заменяется плоским циклом с диспетчером переходов, внедрение мертвого или «мусорного» кода, который никогда не исполняется, но имитирует полезную нагрузку.

4 Противодействие анализу (Anti-Analysis). Включает техники детектирования отладчиков, виртуальных машин и песочниц. Программа может изменять свое поведение или прекращать выполнение при обнаружении среды анализа.

5 Виртуализация кода. Наиболее сложный метод, заключающийся в преобразовании целевого кода в инструкции для кастомной виртуальной машины, реализованной внутри самого приложения. Это делает статический анализ практически невозможным без полного реверса интерпретатора.

2.2 Использованные стратегии обфускации

В разработанном обфускаторе реализованы следующие методы защиты:

1 Переименование идентификаторов. Все имена функций, переменных и аргументов заменяются на трудночитаемые последовательности символов

(например, O0oOabcdef). Это затрудняет понимание логики работы программы и взаимосвязей между её частями.

2 Кодирование строк. Все строковые литералы преобразуются в формат Base64 и декодируются непосредственно во время выполнения. Это предотвращает статический анализ текстовой информации, содержащейся в коде.

3 Сохранение структуры. Обфускатор корректно обрабатывает вложенные структуры и рекурсивные вызовы, сохраняя полную работоспособность исходного алгоритма.

2.3 Реализация обфускатора

Программный продукт реализован в виде класса-трансформера, который проходит по всем узлам синтаксического дерева. Для генерации новых имен используется генератор случайных строк с проверкой на коллизии и исключением зарезервированных слов языка Python (таких как print, int, len и др.).

Особое внимание уделено сохранению корректности f-строк и вызовов встроенных библиотек. Для восстановления строк используется модуль base64, вызов которого автоматически встраивается в начало обфусцированного файла.

На рисунках 2.1 и 2.2 представлен код программы и его обфусцированная версия.

```
fibonacci_mem = [0, 1]


def calculate_nth_fibonacci(n):
    """Calculate the n-th Fibonacci number using recursion."""
    if len(fibonacci_mem) > n:
        return fibonacci_mem[n]
    cur_len = len(fibonacci_mem)
    while cur_len ⩽ n:
        fibonacci_mem.append(fibonacci_mem[cur_len - 2] + fibonacci_mem[cur_len - 1])
        cur_len = len(fibonacci_mem)

    return fibonacci_mem[n]


def main():
    print("Welcome to the Secret Fibonacci Calculator!")
    user_input = 10
    result = calculate_nth_fibonacci(user_input)
    print(f"The result for {user_input} is: {result}")


if __name__ == "__main__":
    main()
```

Рисунок 2.1 – Необфусцированный исходный код

```python
# -*- coding: utf-8 -*-
import base64
O0oOrLKfWNBK = [0, 1]


def O0oOMKuNAsPG(O0oOxhvAARQz):
    base64.b64decode('Q2FsY3VsYXRlIHRoZSBuLXRoIEZpYm9uYWNjaSBudW1iZXIgdXNpbmcgcmVjdXJzaW9uLg==').decode()
    if len(O0oOrLKfWNBK) > O0oOxhvAARQz:
        return O0oOrLKfWNBK[O0oOxhvAARQz]
    O0oOhdbljObk = len(O0oOrLKfWNBK)
    while O0oOhdbljObk ⩽ O0oOxhvAARQz:
        O0oOrLKfWNBK.append(O0oOrLKfWNBK[O0oOhdbljObk - 2] + O0oOrLKfWNBK[O0oOhdbljObk - 1])
        O0oOhdbljObk = len(O0oOrLKfWNBK)
    return O0oOrLKfWNBK[O0oOxhvAARQz]


def main():
    print(base64.b64decode('V2VsY29tZSB0byB0aGUgU2VjcmV0IEZpYm9uYWNjaSBDYWxjdWxhdG9yIQ==').decode())
    O0oOiIBkRKoN = 10
    O0oOhIaFtnDc = O0oOMKuNAsPG(O0oOiIBkRKoN)
    print(base64.b64decode('VGhlIHJlc3VsdCBmb3Ige30gaXM6IHt9').decode().format(O0oOiIBkRKoN, O0oOhIaFtnDc))


if __name__ == '__main__':
    main()
```

Рисунок 2.2 – Обфусцированный исходный код

2.4 Тестирование и результаты

Для проверки работоспособности обфускатора была использована программа, реализующая вычисление n-го числа Фибоначчи. После обработки исходный код стал практически нечитаемым для человека: смысловые названия функций и переменных были утеряны, а все текстовые сообщения заменены на закодированные блоки.

При этом функциональное тестирование показало, что программа сохранила свою исходную логику и выдает идентичные результаты до и после обфускации. Это подтверждает корректность работы реализованных алгоритмов трансформации кода.

На рисунке 2.3 представлены результаты запуска необфусцированной и обфусцированной версий исходного кода.

```
→  acryoz lab7 git:(main ?:2 ✗) python secret_logic.py
Welcome to the Secret Fibonacci Calculator!
The result for 10 is: 55
→  acryoz lab7 git:(main ?:2 ✗) python obfuscated_logic.py
Welcome to the Secret Fibonacci Calculator!
The result for 10 is: 55
```

Рисунок 2.3 – Сравнение работы исходной и защищенной программ

Таким образом были изучены и применены на практике методы обфускации исходного кода.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были освоены практические навыки защиты программного обеспечения от анализа. Было показано, что обфускация является эффективным средством затруднения обратной разработки (reverse engineering) и защиты интеллектуальной собственности в тех случаях, когда исходный код распространяется в интерпретируемом виде.

Реализованный обфускатор продемонстрировал возможность автоматизированного преобразования кода с использованием анализа синтаксических деревьев. Сочетание методов переименования и кодирования данных позволяет значительно повысить порог вхождения для потенциального исследователя кода, при этом не накладывая существенных накладных расходов на производительность программы.

В результате работы были изучены границы применимости обфускации. Несмотря на высокую эффективность против ручного анализа, профессиональные деобфускаторы и средства динамического анализа могут частично восстановить логику программы. Поэтому для критически важных систем обфускация должна применяться в сочетании с другими методами защиты, такими как цифровая подпись кода и контроль целостности среды исполнения.

Все задачи лабораторной работы выполнены в полном объеме, программное средство продемонстрировало стабильную работу и высокую степень запутывания кода при сохранении его функциональной эквивалентности.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Реализация обфускатора

```
import ast
import base64
import random
import string

class Obfuscator(ast.NodeTransformer):
    def __init__(self):
        self.name_map = {}
        self.used_names = set()
        self.excluded_names = {'print', 'range', 'int', 'str', 'len', 'main',
'base64', 'b64decode', 'decode'}

    def _generate_random_name(self, original_name):
        if original_name in self.name_map:
            return self.name_map[original_name]

        if (original_name.startswith('__') or
                original_name in self.excluded_names):
            return original_name

        new_name = 'O0oO' + ''.join(random.choices(string.ascii_letters,
k=8))
        while new_name in self.used_names:
            new_name = 'O0oO' + ''.join(random.choices(string.ascii_letters,
k=8))

        self.used_names.add(new_name)
        self.name_map[original_name] = new_name
        return new_name

    def visit_FunctionDef(self, node):
        if not node.name.startswith('__') and node.name != 'main':
            node.name = self._generate_random_name(node.name)
        for arg in node.args.args:
            arg.arg = self._generate_random_name(arg.arg)
        return self.generic_visit(node)

    def visit_Name(self, node):
        if isinstance(node.ctx, (ast.Load, ast.Store)):
            if node.id not in self.excluded_names:
                node.id = self._generate_random_name(node.id)
        return node

    def visit_JoinedStr(self, node):
        template = ""
        format_args = []
        for value in node.values:
            if isinstance(value, ast.Constant):
                template += str(value.value).replace("{", "{{").replace("}",
"}}")
            elif isinstance(value, ast.FormattedValue):
                template += "{}"
                format_args.append(self.visit(value.value))

        encoded_template = base64.b64encode(template.encode()).decode()
        return ast.Call(
```

```
            func=ast.Attribute(
                value=ast.Call(
                    func=ast.Attribute(
                        value=ast.Call(
                            func=ast.Attribute(
                                value=ast.Name(id='base64', ctx=ast.Load()),
                                attr='b64decode',
                                ctx=ast.Load()
                            ),
                            args=[ast.Constant(value=encoded_template)],
                            keywords=[]
                        ),
                        attr='decode',
                        ctx=ast.Load()
                    ),
                    args=[],
                    keywords=[]
                ),
                attr='format',
                ctx=ast.Load()
            ),
            args=format_args,
            keywords=[]
        )

    def visit_Constant(self, node):
        if isinstance(node.value, str) and not node.value.startswith('__'):
            encoded = base64.b64encode(node.value.encode()).decode()
            return ast.Call(
                func=ast.Attribute(
                    value=ast.Call(
                        func=ast.Attribute(
                            value=ast.Name(id='base64', ctx=ast.Load()),
                            attr='b64decode',
                            ctx=ast.Load()
                        ),
                        args=[ast.Constant(value=encoded)],
                        keywords=[]
                    ),
                    attr='decode',
                    ctx=ast.Load()
                ),
                args=[],
                keywords=[]
            )
        return node
```